PCET's
Pimpri
Chinchwad
University

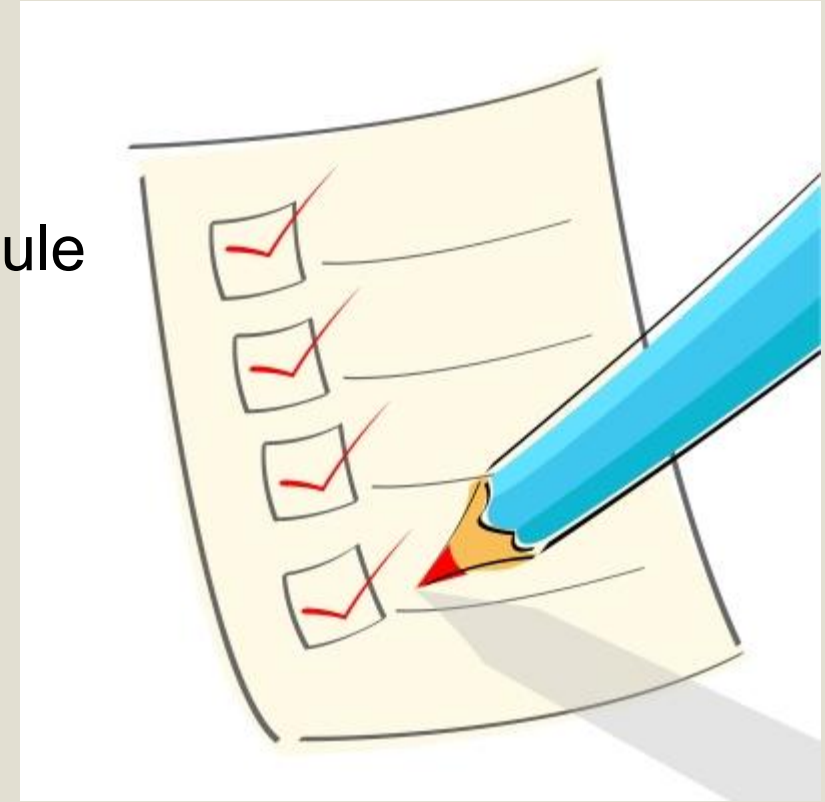
Learn | Grow | Achieve

Module 1:- Introduction to Version control.

Name:- Jaykishan J Saharan
Div:- D
Roll no.:- 27
Batch:- D1

Contents:-

- Introduction
- Short description of each module
- Advantages
- Disadvantages
- Applications
- Conclusion
- GitHub Link
- Certificate



Introduction to version control

Version control is a system that allows multiple people (or even a single developer) to work on a project and track changes to files over time. It is essential for managing source code in software development, but it can be used for any type of document, file, or collaborative project. Version control helps ensure that changes are tracked, allowing you to revert to earlier versions of files, collaborate efficiently, and keep a history of modifications.

Key Concepts in Version Control:

- **Repository**: A repository (or "repo") is a storage location where your project's files are kept. It holds both the current state of the files and the history of changes that have been made to them.
    - **Local Repository**: A copy of the repository that resides on your local machine.
    - **Remote Repository**: A version of the repository hosted on a server, such as GitHub, GitLab, or Bitbucket, which allows collaboration among developers.
- **Commit**: A commit is a snapshot of changes in your project. Every time you save your work, you can commit the changes, which stores those changes with a unique identifier (commit hash). A commit
- includes a message describing the changes made.

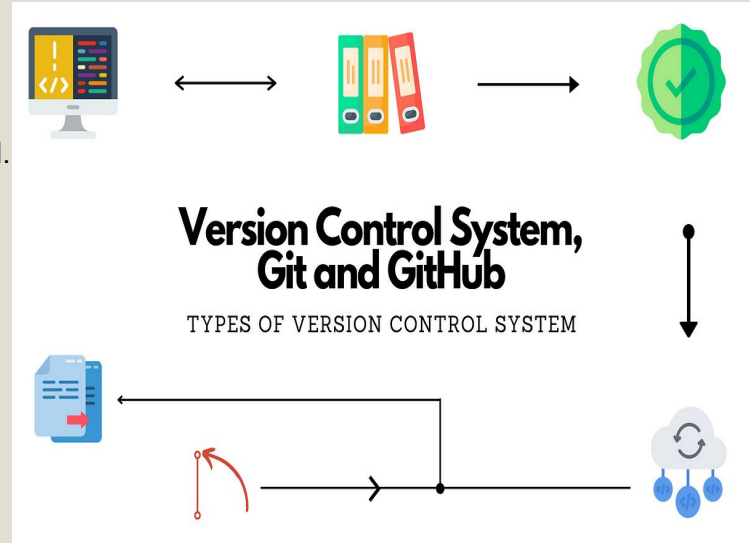
  Example:
    - `git commit -m "Added new feature to homepage"`
- **Branching**: A branch is a copy of the code where you can make changes without affecting the main or stable version of the project. This is especially useful for adding new features or fixing bugs without disturbing the core functionality.
    - **Main branch** (often called `master` or `main`): The primary branch in most version-controlled projects, which should always contain the stable, working version.
    - **Feature branches**: Temporary branches where developers add features or fix bugs before merging them back into the main branch.
- **Merging**: After making changes on a branch, you can merge those changes back into the main branch. This combines your changes with the existing code.
    - Conflicts can arise when two people change the same lines of code. A version control system helps identify and resolve these conflicts.
- **Push and Pull**:
    - **Push**: Pushing is when you upload your local changes to the remote repository (e.g., GitHub). This is typically done after you commit your changes locally.
        - `git push origin branch-name`
    - **Pull**: Pulling is the process of downloading the latest changes from the remote repository to your local machine.
        - `git pull origin branch-name`
- **History and Versioning**: Version control keeps a complete history of all changes made to the project, which allows you to:
    - View who made specific changes.
    - Revert back to previous versions.
    - Track when specific changes were made and why.
    - Compare different versions of a file to see what has changed.

Types of Version Control:

Local Version Control: In this approach, a single database on a local machine keeps track of changes to files. It's simple but not ideal for collaboration or large teams.

Centralized Version Control (CVCS): With centralized version control systems like **Subversion (SVN)**, there's a central server that contains the repository. Every developer gets their own working copy of the repository, but the history of all changes is stored in one central location.

- Pros: Easy to set up, centralized control.
- Cons: If the central server goes down, nobody can work or commit changes.



Distributed Version Control (DVCS): Systems like **Git** and **Mercurial** are distributed. In this model, each developer has a full copy of the entire repository, including its history. This allows them to work independently of a central server and still have a c omplete version history.

- Pros: Better for collaboration, faster performance, more resilient to failures.
- Cons: Slightly more complex setup and operation.

Git is the most popular DVCS and has become the standard for modern software development. Services like **GitHub**, **GitLab**, and **Bitbucket** provide cloud-hosted Git repositories with additional tools for collaboration.

Key Benefits of Version Control / Advantages:-

Collaboration: Version control allows multiple developers to work on the same project simultaneously without overriding each other's work. Changes are tracked and can be merged later.

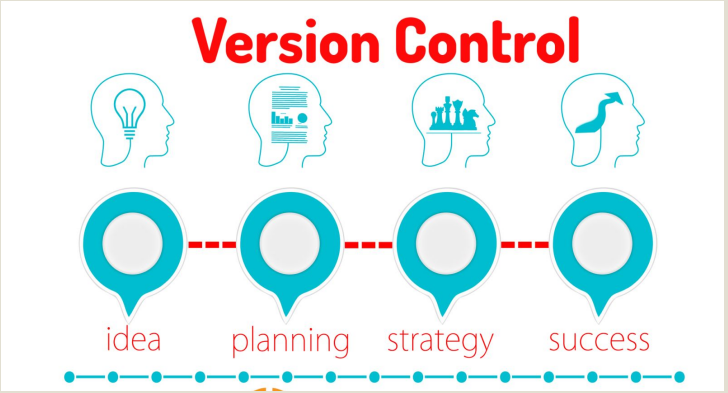
Backup: Since every change is saved and backed up, you can always restore the project to a previous version if something goes wrong.

Traceability: Version control allows you to track exactly who made what changes and why, which is valuable for auditing and debugging.

Branching and Experimentation: You can create branches to try out new features or make risky changes without affecting the main project. Once the changes are stable, they can be merged into the main branch.

Conflict Resolution: In case multiple people edit the same file, version control systems help detect and resolve conflicts.

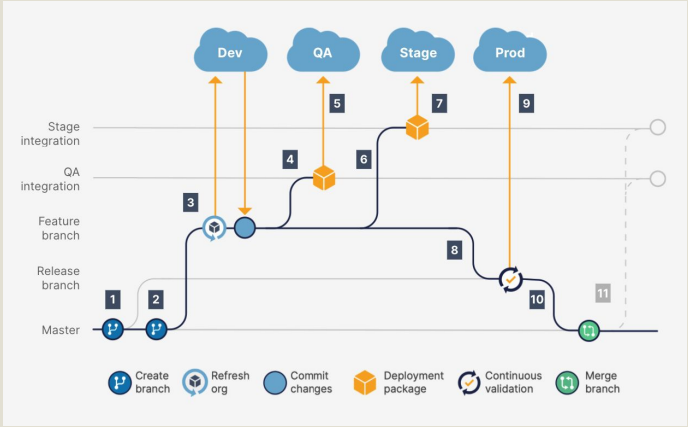
History: You have a complete record of the project's history, which is useful for debugging, understanding the evolution of the project, and identifying when specific changes were introduced.



Demerits of version control / Disadvantages :-

Complexity and Learning Curve:

- **Initial Setup**: For beginners, understanding version control can be overwhelming. Tools like Git have a steep learning curve, especially when it comes to concepts like branching, merging, rebasing, and resolving conflicts.
- **Advanced Features**: Features like rebasing, cherry-picking, or resolving complex merge conflicts may confuse new users or even experienced developers if not used properly.



Overhead for Small Projects:

Storage and Performance Issues:

Merge Conflicts
:

Maintenance Overhead:

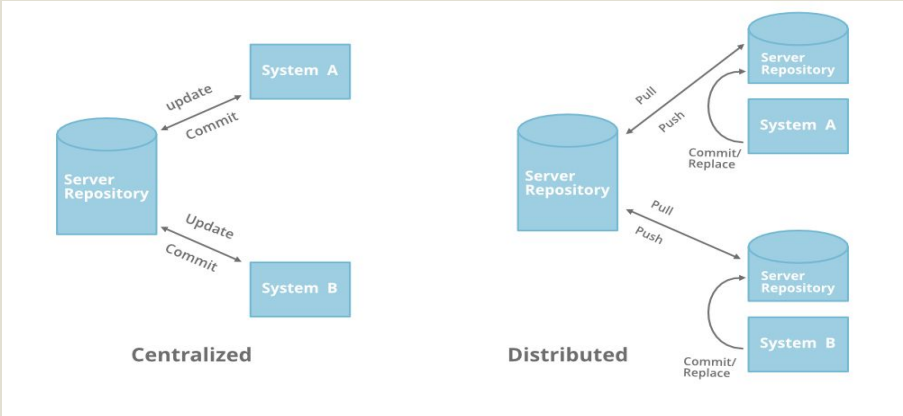
Collaborative Issues:

Security Risks:

Dependency on Centralized Services:

Conflict with Large Teams or Projects:

Overcommitment or Poor Commit Practices:



Applications of version control:-

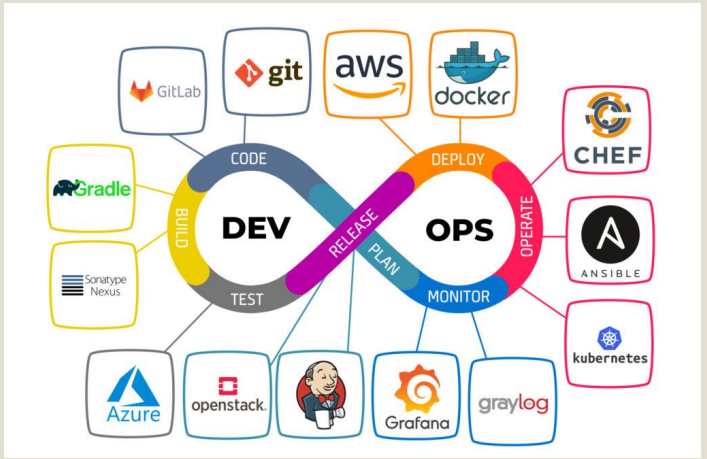
Popular Version Control Systems:

1. **Git**:
   - The most widely used version control system, particularly in open-source projects.
   - Distributed, fast, and efficient.
   - Works well with Git hosting platforms like GitHub, GitLab, and Bitbucket.

2. **Subversion (SVN)**:
   - A centralized version control system.
   - Less popular than Git but still used in some enterprise environments.

3. **Mercurial**:
   - A distributed version control system similar to Git.
     Less popular than Git but still used in some projects.

4. **Perforce**:
   - Often used in large enterprises and game development due to its ability to handle large files and complex codebases.



Examples:-

1. **Initialize a Git Repository**: To create a new Git repository in a project folder:

```bash
git init
```

2. **Add Files to the Repository**: Stage files for commit:

```bash
git add .
```

3. **Commit Changes**: Commit the staged files with a message describing the changes:

```bash
git commit -m "Initial commit"
```

4. **Push to a Remote Repository**: Push changes to a remote repository (e.g., GitHub):

```bash
git push origin main
```

5. **Pull Changes**: Pull the latest changes from a remote repository:

```bash
git pull origin main
```

6. **Create a Branch**: Create a new branch for a feature:

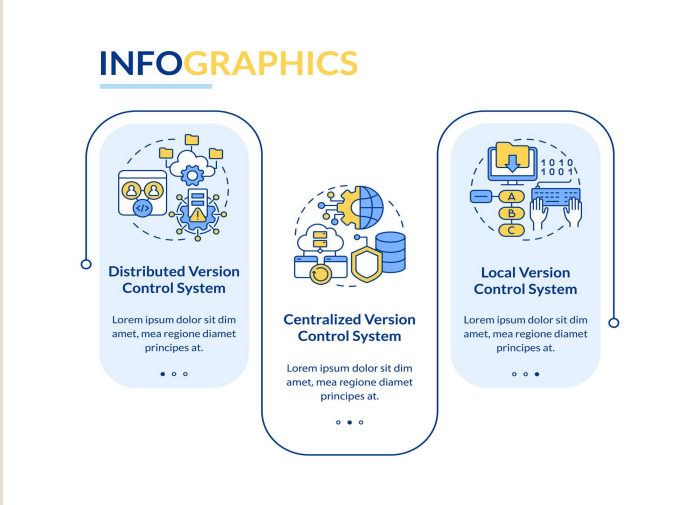
```bash
git checkout -b feature-branch
```

7. **Merge Branches**: After completing work on a branch, merge it back into the main branch:

```bash
git checkout main
git merge feature-branch
```

- Conclusion

Version control is an essential tool for developers, enabling collaboration, ensuring project integrity, and providing a reliable history of changes. Git, being the most popular version control system, allows developers to manage their code effectively, track changes, and maintain a clean development workflow. Understanding version control is a fundamental skill for anyone involved in software development, whether working individually or as part of a team.



certificate